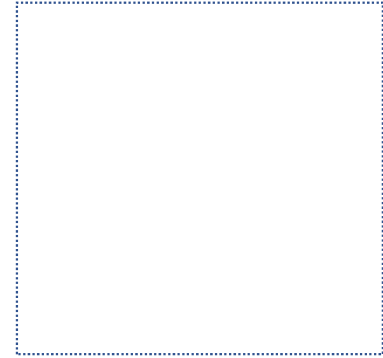
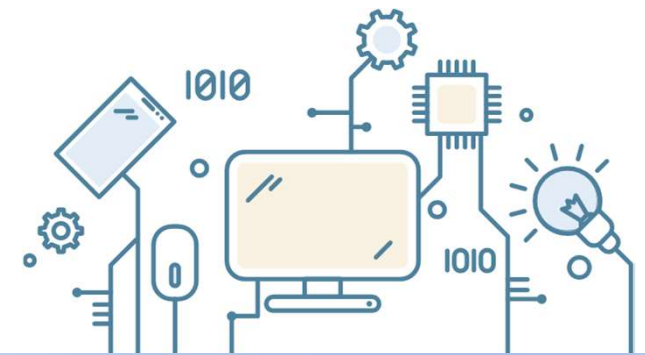


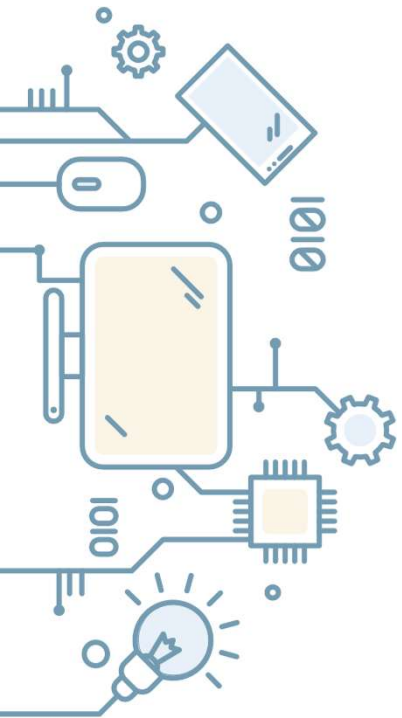
- 본 자료에 있는 그림의 일부는 “4차 산업 혁명 시대의 이산수학”의 저자 및 출판사의 허락을 받고 사용하고 있으므로 본 강의 수강이외의 목적으로 무단 복제 및 배포를 금합니다.

Graph Theory & Its Applications



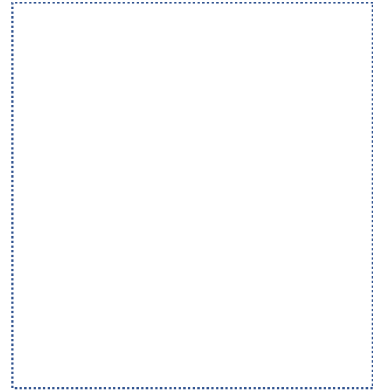
홍 기 형





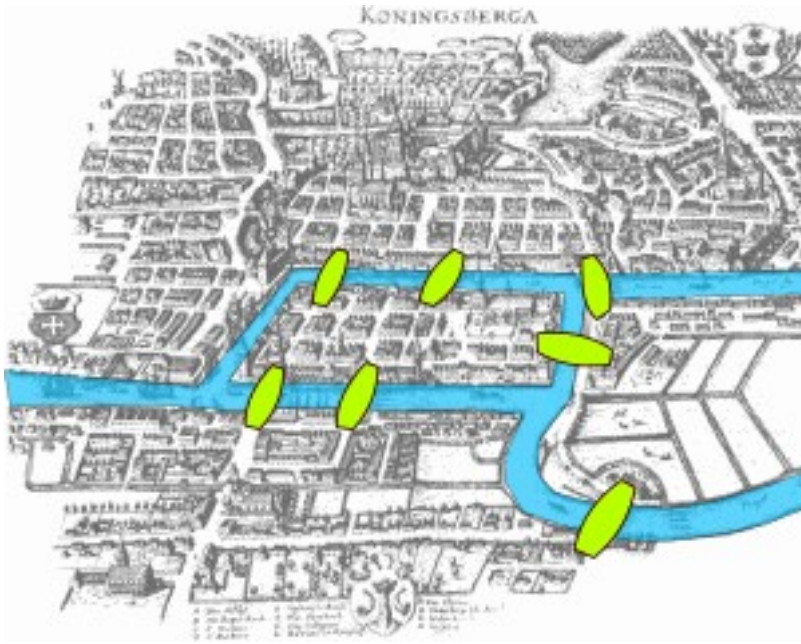
목차

1. 그래프의 기본 개념
2. 그래프의 용어
3. 그래프의 표현
4. 특수 형태의 그래프
5. 그래프의 응용
6. 그래프의 탐색
7. 그래프와 색칠 문제 개요



그래프(Graph)의 기본 개념

- 18세기 스위스 출신의 저명한 수학자 오일러(Leonhard Euler, 1707~1783)에 의해 그래프 이론이 본격적으로 시작됨
- 그래프 이론의 대표적인 예인 쾨니히스베르크(Königsberg) 다리 문제는 두 개의 섬과 강둑 사이를 연결하는 7개의 다리가 있을 때 각 다리를 꼭 한 번씩만 건너서 출발한 장소로 돌아오는 경로를 찾는 문제임



그래프 응용분야

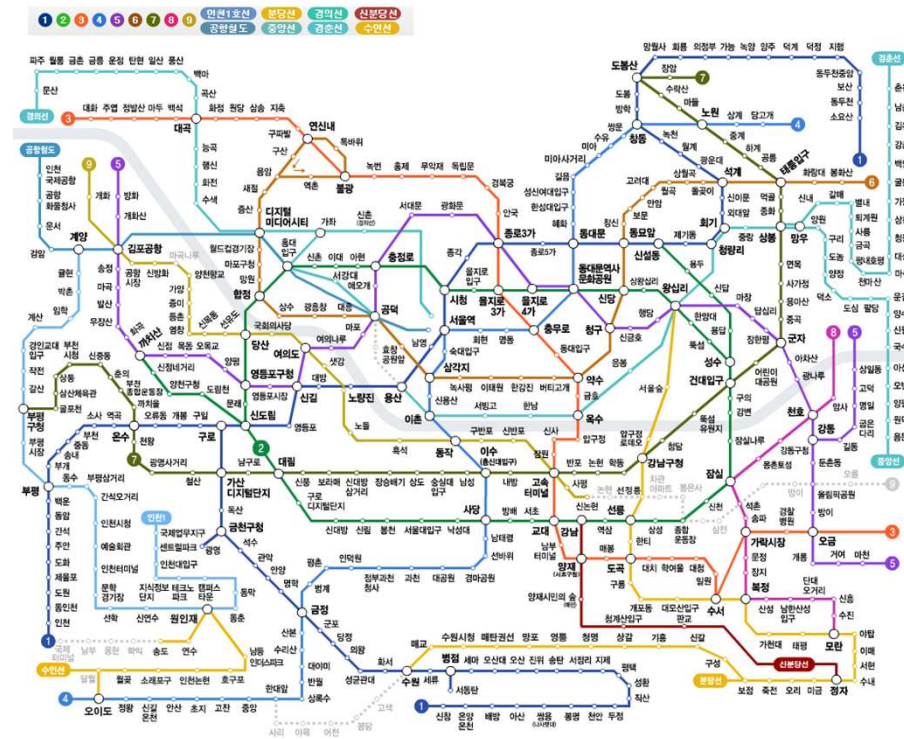


1010LAB

- 다양한 망 설계 분야
 - 도로망, 철도망, 통신망(network), 전기회로망 등
 - 연결성과 최적 비용 계산 – 최단거리 순회
- 길 찾기, 경로 찾기
 - 두 지점 사이의 길(경로)
 - 그래프 탐색, 최단경로, 최적 경로
- 다양한 문제 풀이
 - 그래프 모델링과 자료구조로 활용
 - 알고리즘 설계
 - 최적화, 작업계획분석 및 일정 계획
- Finite state machine (유한상태 기계)
- 유전, 언어, 사회과학, 자연과학 등 다양한 분야에서 응용되고 있음.

그래프의 정의

- $G = (V, E)$
 - V : a finite set of vertices (nodes, 정점)
 - E : a set of edges (연결선)
 - edge : a pair of two vertices



방향/무방향 그래프

- 방향 그래프(directed graph 또는 digraph)
 - 방향이 있는 그래프임
 - 연결선을 **화살표**로 표시하여 방향을 나타내는 그래프임
- 방향이 없는 그래프(undirected graph)
 - 방향이 없는 그래프임
 - 그래프의 특수한 형태이므로 특별한 언급이 없는 한 그래프는 방향이 없는 그래프를 의미함

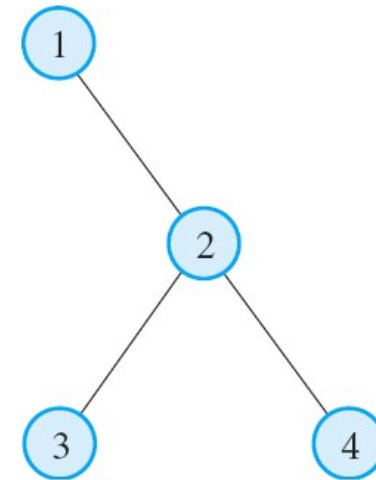
그래프 (무방향) 예



1010 LAB

- 그래프의 간단한 예

- $V = \{1, 2, 3, 4\}$ 이며 $E = \{(1, 2), (2, 3), (2, 4)\}$



〈그림 7.2〉 그래프의 간단한 예

그래프 용어



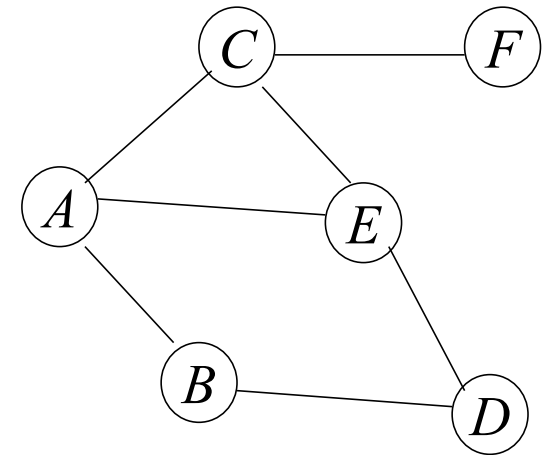
1010 LAB

- **경로 (path)**

- 모든 $1 \leq i < k$ 에 대해 연결선 (v_i, v_{i+1}) 이 존재할 때, 정점들의 열(sequence) $v_1, v_2, v_3, \dots, v_k$ 라고 함
- a sequence of edges (or vertices) from a vertex to another vertex
- $k \geq 1$ 이며 이 경로의 길이는 $k - 1$ 임
- A에서 F로의 경로 : (A, C, F) , (A, B, D, E, C, F) , $(A, C, E, A, C, F), \dots$

- **사이클 (cycle)**

- $v_1 = v_k$ ($k \neq 1$)이면 이러한 경로를 사이클이라고 함
 - a path in which the first and last vertices are the same and no edges are repeated.
- 사이클의 예: (A, E, D, B, A) , ...
- 사이클이 아닌 예: (A, C, E, C, A)



방향 그래프 (Directed Graph, Digraph)

- $G = (V, E)$

- V : a finite set of vertices (nodes, 정점)

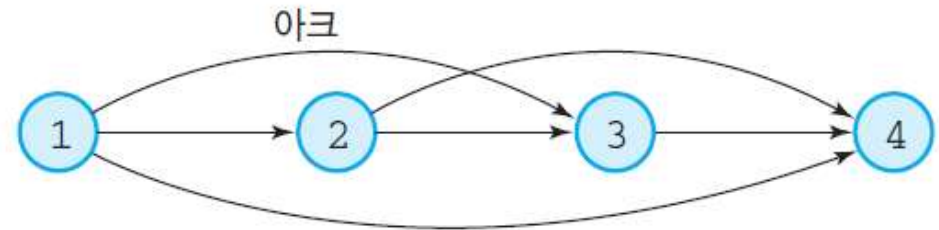
- E : a set of arcs (아크, 화살표)

- arc : an ordered pair of two vertices,
- 정점 v 에서 w 로 가는 아크, $v \rightarrow w$

- 방향 그래프의 예

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ 는 정점 1에서 정점 4로 가는 경로임

- $v \rightarrow w$ 인 아크에 대하여 v 를 w 의 선행자(predecessor)라 하며 w 를 v 의 후속자(successor)라고 함

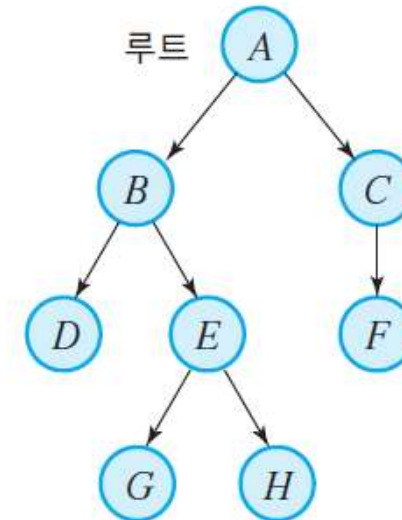


〈그림 7.3〉 방향 그래프($\{1, 2, 3, 4\}, \{v \rightarrow w \mid v < w\}$)

트리 (Tree)

- 사이클(cycle)이 존재하지 않는 그래프임
- 루트(root)라는 특별한 노드가 한 개 존재하고, 루트로부터 다른 모든 노드로 가는 경로가 항상 유일하게 존재함
- 루트로 들어오는 연결선이 없으므로 루트는 모든 트리의 출발점이 됨

- 무방향그래프로도 표현함. (단 위 조건은 동일)
- 부모자식 관계(parent-child relationship)



〈그림 7.4〉 트리

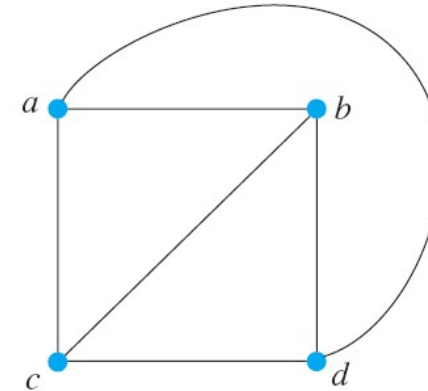
단순/멀티그래프

- 단순 그래프 (simple (undirected) graph)

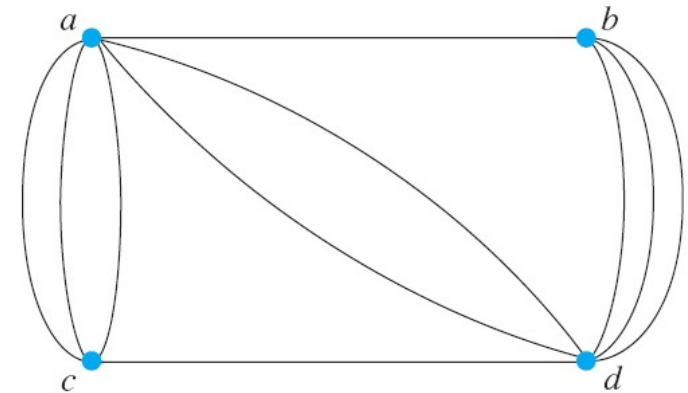
- 한 쌍의 정점 사이에 최대 1개의 연결선 존재 (at most one edge between two vertices)
- no self-cycle (self-cycle: 자기 자신으로의 연결선)
- $G = (V, E)$
 - V : a finite set of vertices
 - E : an irreflexive, symmetric, relation on V
 - A set of edges
 - Irreflexive : there are no edges from a vertex to itself
 - Symmetric : $(u,v) \in E$ iff $(v,u) \in E$

- 멀티 그래프 (multigraph)

- there is no limitation in the number of edges between edges.



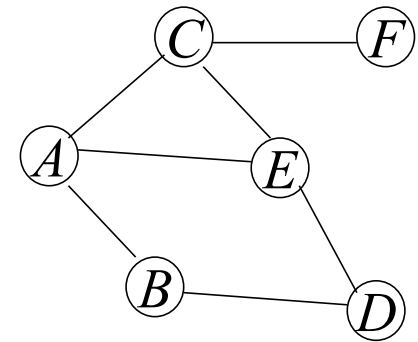
〈그림 7.5〉 단순 그래프의 예



〈그림 7.6〉 멀티 그래프의 예

인접 (adjacency)

- **Adjacency :**
 - two vertices are adjacent to each other if they are connected by a single edge.
 - If $(u,v) \in E$, u is adjacent to v . (u 와 v 는 서로 인접하다.)
- **Degree of a vertex v , $d(v)$ or $\deg(v)$: v 의 차수**
 - the # of edges incident to v (인접하는 또는 접하는 연결선의 수)
 - $\deg(A)=3, \deg(F)=1$
- **Theorem: Let $G=(V,E)$.**
$$\sum_{v \in V} \deg(v) = 2 |E|$$

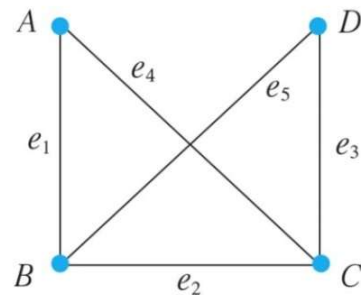


$$\sum_{v \in V} \deg(v) = 2 |E|$$



예제 7-6

다음 그래프에서 모든 정점들의 차수의 합은 연결선들 개수의 2배임을 보이자.



풀이 $\deg(A) = 2, \deg(B) = 3, \deg(C) = 3, \deg(D) = 2$

이므로, 차수의 합은 10이고 연결선은 5개이므로 차수의 합은 연결선 수의 두배와 같다.

부분 그래프, 생성 부분 그래프, 완전 그래프

- 부분 그래프 (subgraph)

- A subgraph of $G=(V,E)$ is a graph $G'=(V',E')$,

where $V' \subseteq V$ and $E' \subseteq E$

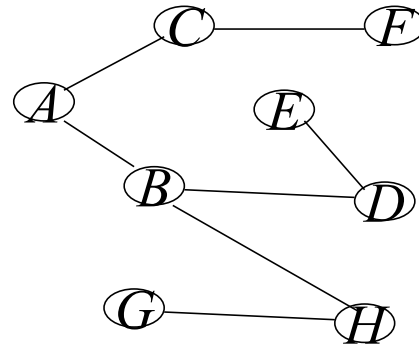
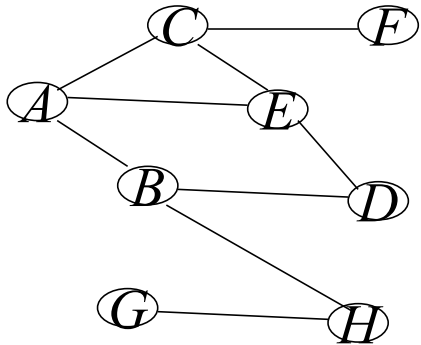
- 생성부분 그래프 (spanning subgraph)

- A subgraph where $V' = V$

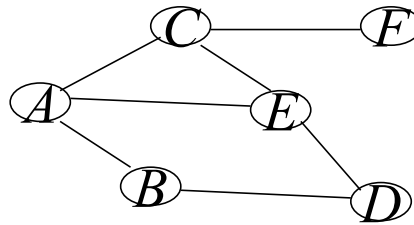
- 완전그래프 (complete graph)

- A graph $G=(V,E)$ is *complete* if for all vertices $u,v \in V$, $(u,v) \in E$.
- A complete graph with n vertices: K_n

부분, 생성부분, 완전 그래프 예

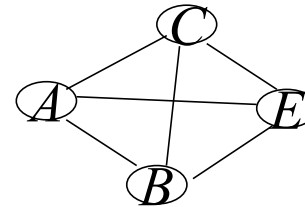
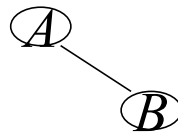


A spanning subgraph

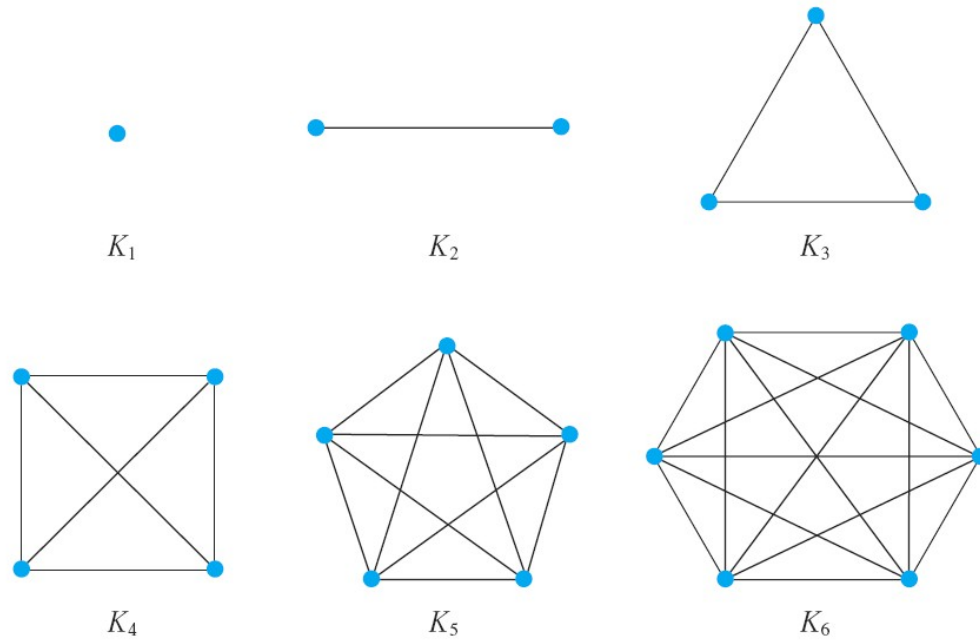


Two subgraphs

complete graphs when $|V|=2$ and $|V|=4$



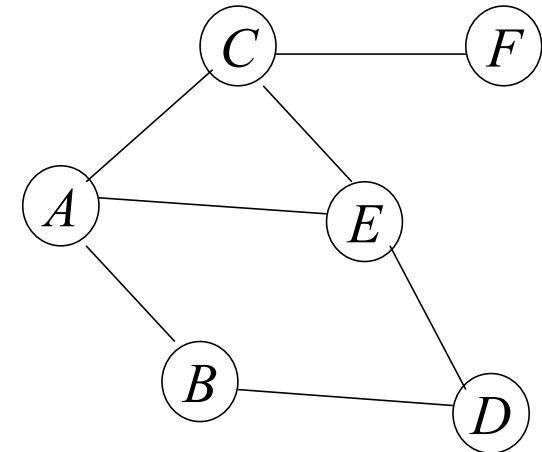
완전 그래프 예



〈그림 7.13〉 완전 그래프의 예

경로 관련 용어

- 경로(path)
 - 꼭지점의 열 $v_1, v_2, \dots, v_n$ 에서 $(v_{k-1}, v_k) \in E, 1 \leq k < n$ 인 경우를 말함
 - 경로의 길이(length)는 $n-1$ 임
 - (연결선의 sequence로도 표현함)
- 단순 경로(simple path)
 - 경로가 같은 연결선을 두 번 포함하지 않는 경로를 말함
 - A,E,C,F A,E,D,B,A
 - A,E,C,E,D (X)
- 기본 경로(elementary path)
 - 어떤 정점들도 두 번 만나지 않는 경로를 말함
 - A,E,C,F
 - A,E,C,E,D (X), A,E,D,B,A (X)

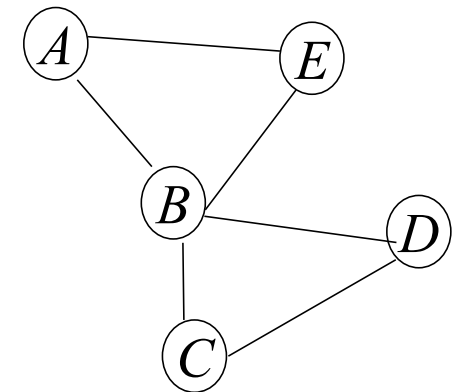
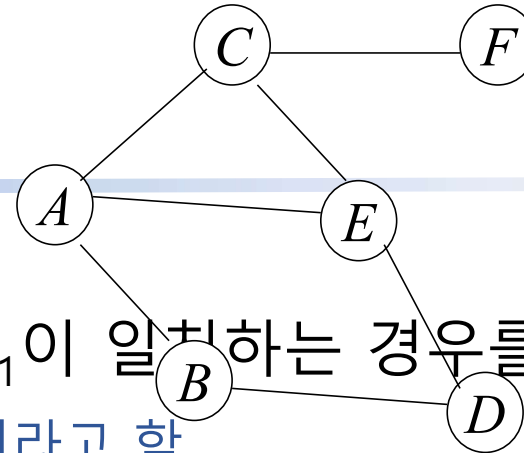


사이클 관련 용어



1010 LAB

- 사이클(cycle) 또는 순회(circuit)
- 경로 $(v_1, v_2, \dots, v_n)$ 에서 종점 v_n 과 시점 v_1 이 일치하는 경우를 말함
 - $v_1 = v_k$ ($k \neq 1$)이면 이러한 경로를 사이클이라고 함
 - a path in which the first and last vertices are the same and no edges are repeated.
- 단순 사이클(simple cycle)
 - 같은 연결선을 반복하여 방문하지 않는 사이클을 말함
 - 단순 사이클의 예: (A, E, D, B, A) , ...
 - 단순 사이클이 아닌 예: (A, C, E, C, E, A)
- 기본 사이클(elementary cycle)
 - 시작점을 제외한 어떠한 정점도 반복하여 방문하지 않는 사이클을 말함



연결 그래프



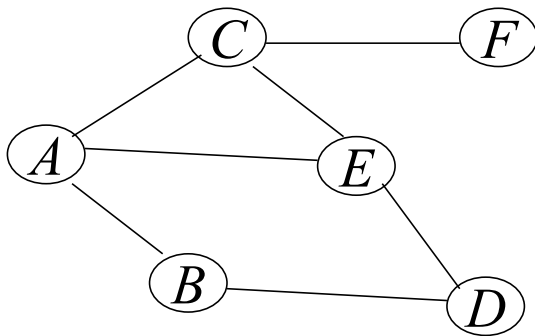
1010 LAB

- 연결 (connected)
 - two vertices are **connected** if there is a path between them
- 연결 그래프(connected graph)
 - 그래프의 모든 정점들이 연결되어 있는 그래프임
 - **there is at least one path from every vertex to every other vertex**
- 강한 연결 그래프(strongly connected graph only in digraphs)
 - 그래프에서 모든 두 정점 a와 b에 대해서 a에서 b로의 경로와 b에서 a로의 경로들이 존재하는 그래프를 말하는데, 특히 방향 그래프에서만 의미를 가짐
- 연결 요소(connectivity component)
 - 그래프에서 모든 정점들이 연결되어 있는 부분을 말하며, 연결 수(connectivity number)란 그래프 G에서의 연결 요소의 개수를 말함
 - **Connectivity Number, $C(G)$: number of connected components in a graph G**
 - **$C(G) = 1$ iff G is a connected graph.**

연결 그래프 예



1010 LAB



$G' = (\{A, B, C, D, E, F\}, E')$
: connected
 $C(G') = 1$



$G = (\{A, B, C, D, E, F, G, H\}, E)$
: not connected
 $C(G) = 2$

연결 그래프의 예

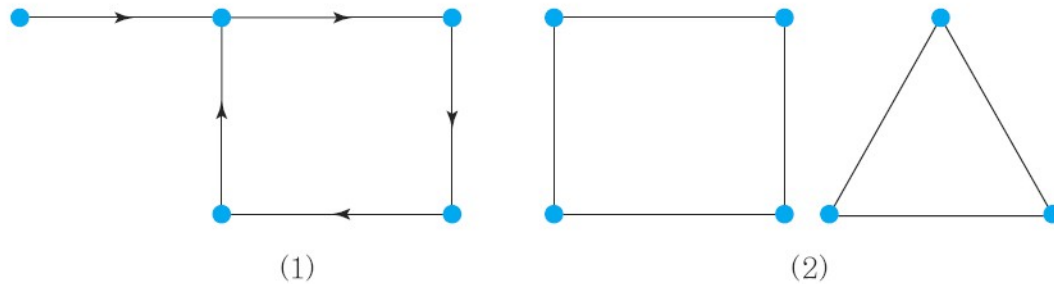


1010 LAB



예제 7-2

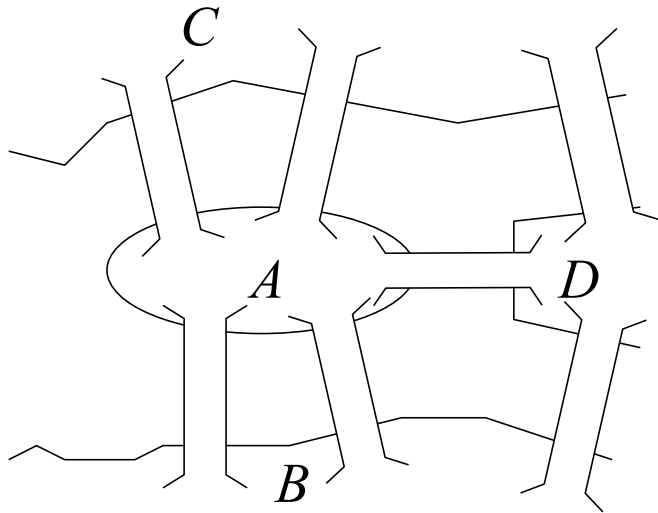
다음 그림이 연결 그래프인지 강한 연결 그래프인지를 판단해보자.



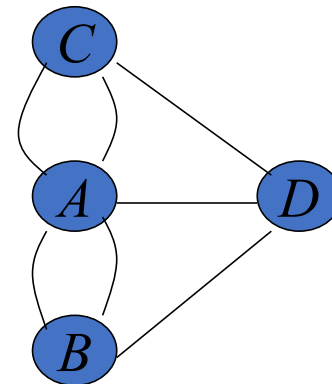
풀이 (1)은 연결 그래프이나 강한 연결 그래프는 아니다. 왜냐하면 연결선의 반대 방향으로의 경로가 없기 때문이다. (2)는 그래프가 서로 떨어져 있기 때문에 연결 그래프가 아니다.

멀티 그래프 (Multigraph)

- 멀티 그래프(Multi-graph)란 한 쌍의 정점 사이에 연결선의 개수의 제한이 없는 중복된 연결선을 허용하는 특별한 그래프임
- 쾨니히스베르크 다리 문제를 해결하는 방법은 멀티 그래프를 모델링하는 것임

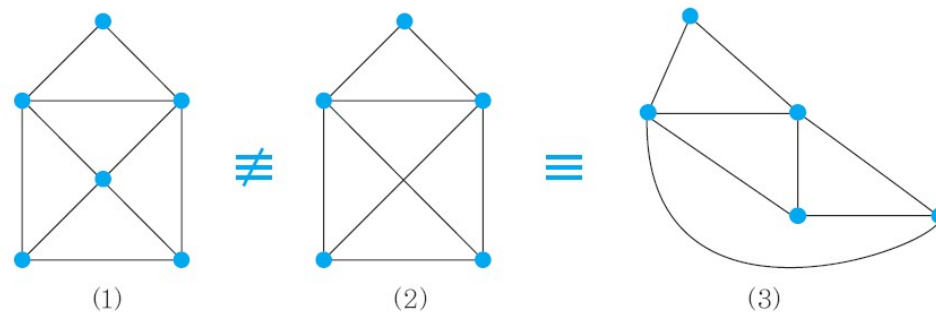


*Bridges of
Königsberg*



그래프의 위상 동일성 (Isomorphisms)

- 기하학에 위상적인 개념을 접합시킨 응용으로서, 위상 기하학(Topological Geometry)적인 관계를 나타냄
- 그래프에서 점의 위치나 선의 길이 등에는 특별한 의미를 부여하지 않고 위상적인 형태에만 중점 둠
- 아래 그림의 (1)과 (2)는 위상적으로 서로 다르지만, (2)와 (3)은 위상적으로 같음



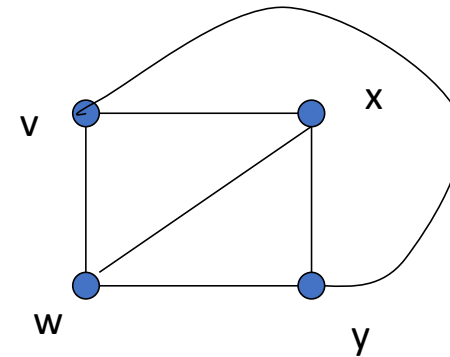
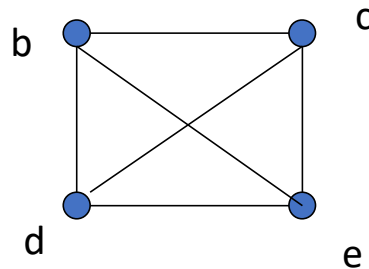
〈그림 7.8〉 위상기하학적 관계

위상 동일성

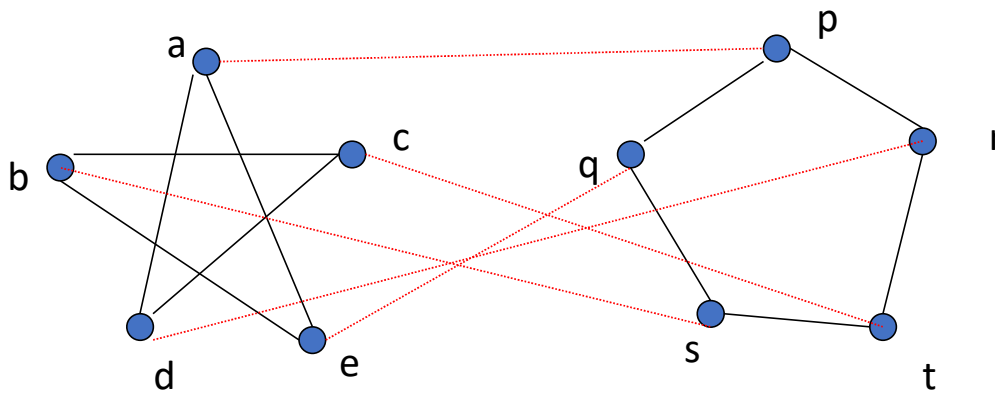


1010 LAB

- Two objects have the same structure, they are isomorphic.
- $G1=(V1,E1)$ and $G2=(V2,E2)$
- An isomorphism from $G1$ to $G2$
 - A bijection $f: V1 \rightarrow V2$, such that
for all $u,v \in V1$, $(u,v) \in E1$ iff $(f(u), f(v)) \in E2$.



- Show that two graphs are isomorphic.



v	a	b	c	d	e
f(v)	p	s	t	r	q



(u,v)	(f(u),f(v))
(a,d)	(p,r)
(a,e)	(p,q)
(b,c)	(s,t)
(b,e)	(s,q)
(c,d)	(t,r)

그래프의 표현



1010LAB

- **그래프의 표현 방법**

- 그래프는 그림을 이용하여 표현하는 것으로 가장 자연스럽게 이해하기에도 가장 쉬운 방법임
- 컴퓨터는 그림으로 표현된 정보를 이용할 수 없기 때문에 통상 **인접 행렬**이나 **인접 리스트**에 의해 표현됨
- 이를 통하여 컴퓨터 프로그램으로 구현됨

인접 행렬 (Adjacency Matrix)

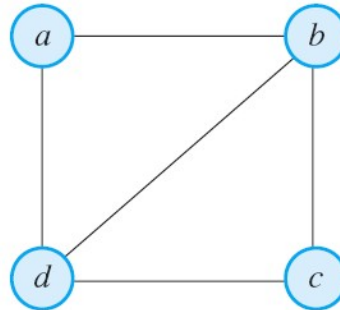
- Adjacency Matrix (2 dimensional array)
 - 그래프 $G = (V, E)$ 에서 $|V| = n$ 일 때 G 의 인접 행렬은 $n \times n$ 행렬 A 로 나타내어지며, 이때 A 의 각 원소 a_{ij} 는 다음과 같이 정의됨
 - $n \times n$ matrix for graph having n vertices
 - $a_{ij} = 1$, if vertex j is adjacent to vertex i (i.e., $(v_i, v_j) \in E$).
 $= 0$, otherwise

인접 행렬 예



예제 7-3

다음과 같은 그래프의 인접 행렬을 구해보자.



풀이 정점의 개수가 4개이므로 다음과 같은 4×4 행렬로 만들어진다.

$$\begin{array}{c} a \quad b \quad c \quad d \\ \begin{array}{l} a \\ b \\ c \\ d \end{array} \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

인접 행렬 예

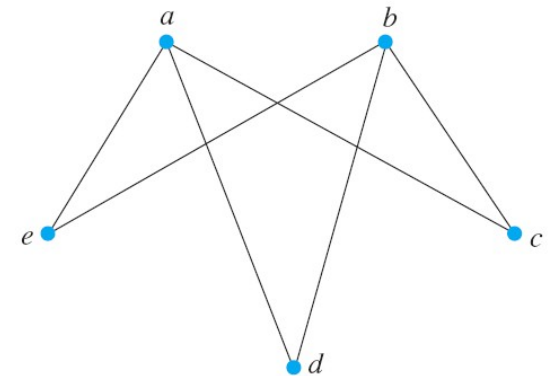


예제 7-4

다음과 같은 인접 행렬을 가지는 그래프를 그려보자.

$$\begin{array}{c} a \quad b \quad c \quad d \quad e \\ \begin{array}{l} a \\ b \\ c \\ d \\ e \end{array} \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

풀이 인접 행렬은 그래프에서 두 꼭지점 사이에 변이 있으면 1로, 그렇지 않으면 0으로 나타내므로, 그것에 따라 다음과 같은 그래프를 그릴 수 있다.

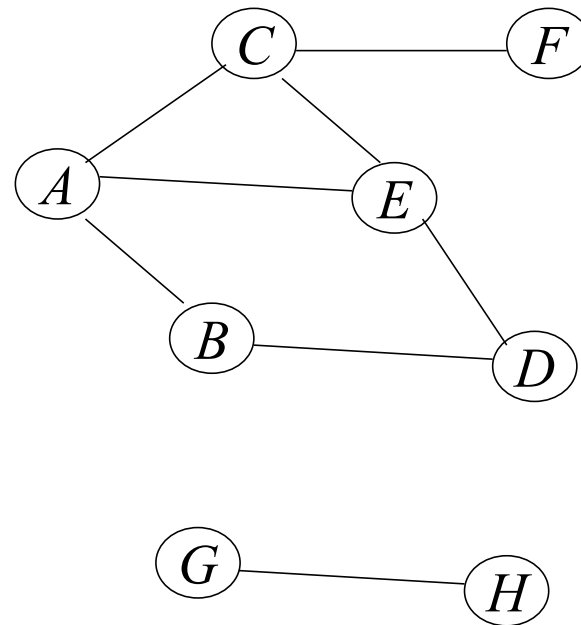


인접 행렬 예

	0	1	2	3	4	5	6	7
0	0	1	1	0	1	0	0	0
1	1	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0
3	0	1	0	0	0	0	0	0
4	1	0	1	1	0	0	0	0
5	0	0	1	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	1	0

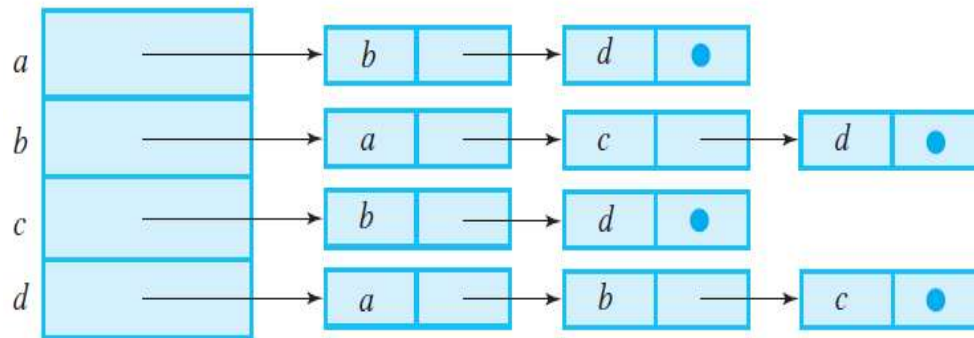
Lower triangular matrix

A : 0
B : 1
C : 2
D : 3
E : 4
F : 5
G : 6
H : 7

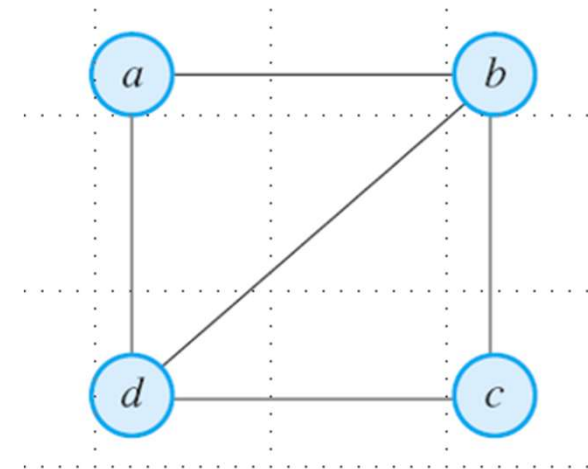


인접리스트 (Adjacency List)

- 각 정점에 대해 **포인터(pointer)**가 주어지고, 그 점으로부터 연결된 정점들을 차례로 **연결 리스트(linked list)**로 표시함
- 같은 리스트 내에서는 순서에 관계가 없음



〈그림 7.9〉 인접 리스트

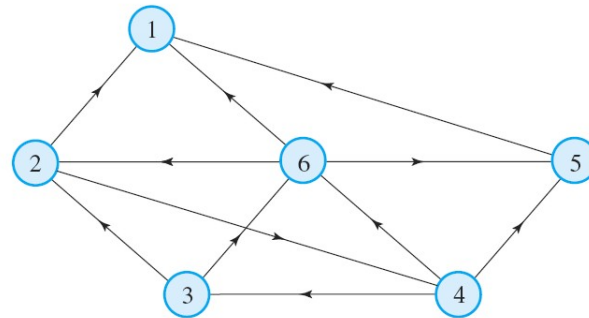


방향 그래프의 인접 리스트

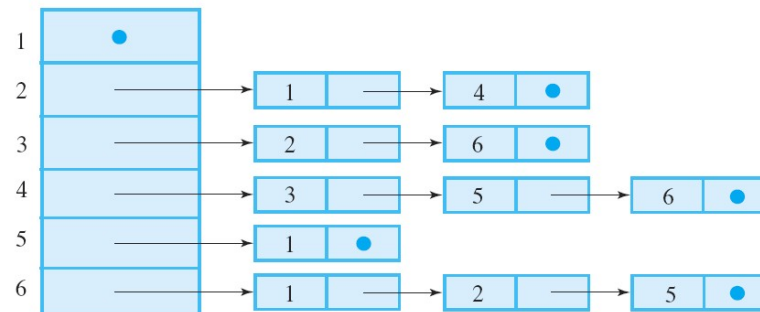


예제 7-5

다음의 방향 그래프에서 인접 리스트를 구해보자.



풀이 여기서는 정점 1에서 나가는 연결선이 없다.

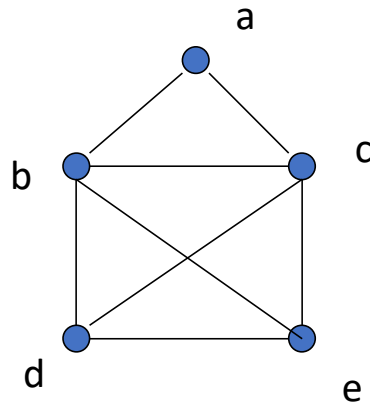


특수형태의 그래프

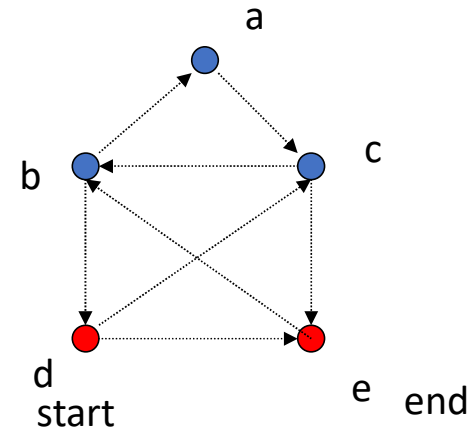
- 오일러 경로/순회
- 해밀턴 경로/순회
- 가중 그래프
- 완전 그래프
- 이분 그래프
- 동형 그래프 (isomorphism)
- 평면 그래프
- 방향 비사이클 그래프

오일러 경로/순회 (Eulerian Path/Circuit)

- 오일러 경로 (Eulerian Path)
 - A path that uses all edges in a multigraph exactly once.
 - 모든 vertex를 방문하면서 각 edge는 단 한번만 모두 지나가는 path
- 오일러 순회 (Eulerian circuit)
 - 오일러 경로 중에서 simple cycle인 것



Eulerian path



오일러 경로/순회의 필요 충분조건

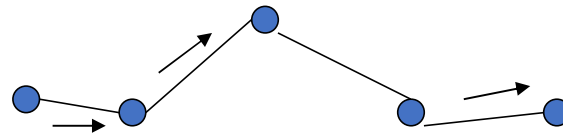
• Theorem:

- A connected graph with at least 2 vertices has an Eulerian path iff there are 0 or 2 vertices of odd degree. (한 붓 그리기)
- The path is a cycle iff each vertex has even degree.

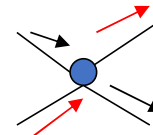
• proof

- If there exists a Eulerian path, then there are 0 or 2 vertices of odd edges.

- 출발 vertex와 마지막 vertex를 제외한 path 상의 중간에 있는 vertex는 들어오는 edge가 있으면, 그에 쌍이 되는 나가는 edge가 존재해야 한다.



- 만일 path가 cycle이면, 출발 vertex도 even edges를 가져야 한다.

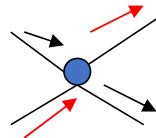


proof (continue)



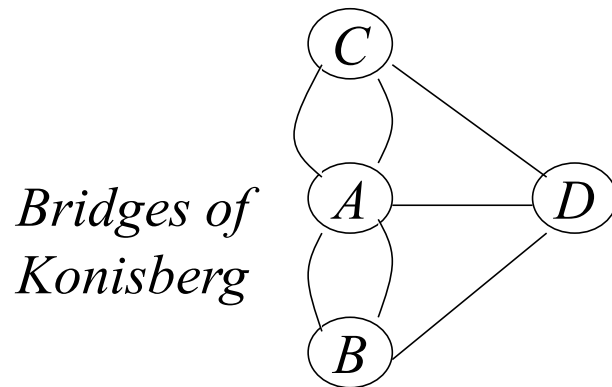
1010 LAB

- If there are 0 or 2 vertices of odd edges, then there exists a Eulerian path.
 - 모두 even edges: 임의의 vertex를 출발점으로 해서 여기서 edge를 하나 선택하여, 인접한 vertex로 이동, 이동한 뒤 사용한 edge를 제거하면, 남은 edge는 odd이고 돌아 오는 edge를 제외하면, 다시 짝수 개 edge가 남음.
 - v 에서 출발하여 v 에서 끝나는 path를 만들면, cycle이 형성됨.
 - 2개 vertices가 odd edges: 이들 중에 하나를 출발지로, 다른 하나를 마지막 vertex로 하면, path 존재.



코니히스베르크 다리 문제 → 오일러 경로/순회

- 멀티 그래프에서 모든 연결선들을 꼭 한 번씩만 통과하는 경로를 **오일러 경로(Eulerian path)**라고 하는데, 코니히스베르크 다리 문제는 오일러 경로를 찾을 수 있는지 여부와 동치임
- 멀티 그래프에서의 오일러 경로를 판별하는 규칙은 모든 정점에서 그것과 연결된 연결선의 개수가 홀수인 정점(**홀수점**)의 개수가 0 또는 2개인 경우임
- 그래프에서는 A, B, C, D 4개의 정점들이 모두 홀수점을 가지므로(총 4개) 오일러 경로가 없다. 따라서 각 다리를 꼭 한 번씩만 건너는 경로가 존재하지 않음

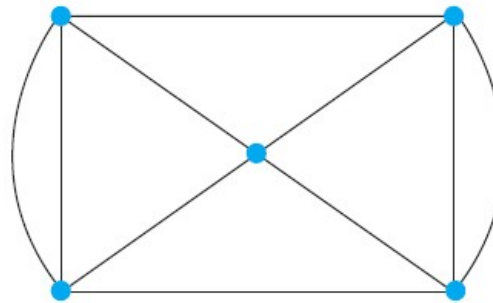


오일러 경로 예



예제 7-8

다음 그래프는 한붓그리기가 가능한지를 판별해보자.

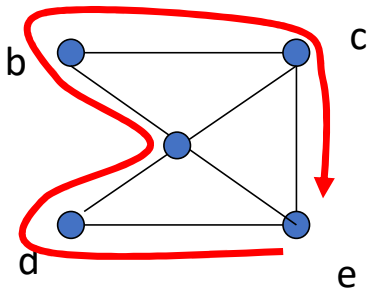


풀이 각 정점의 차수가 모두 짝수 차수로서 홀수점이 존재하지 않는 오일러 경로이므로 한붓그리기가 가능하다.

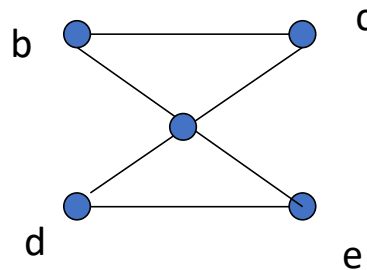
오일러 순회는?

해밀턴 경로/순회

- 해밀턴 경로 (Hamiltonian path) :
 - A path that uses all vertices in a graph exactly once.
- 해밀턴 순회 (Hamiltonian circuit) :
 - a cycle that passes through each vertex exactly once
 - A graph with a Hamiltonian circuit is connected, but not every connected graph has a Hamiltonian circuit.



Graph having a Hamiltonian circuit



Graph having no Hamiltonian circuit

해밀턴 순회 예

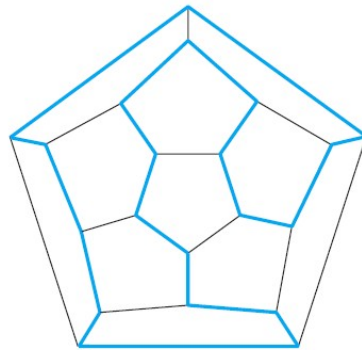


1010 LAB



예제 7-9

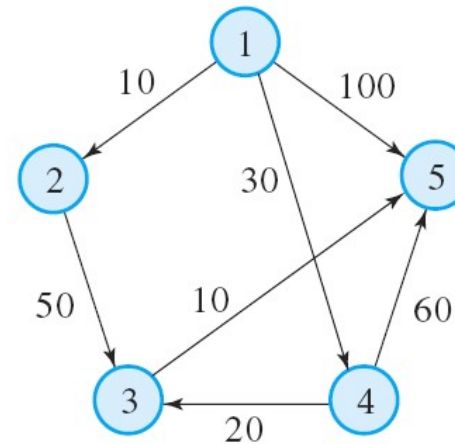
다음 그래프에서 굵은 색선으로 표현된 연결선은 해밀턴 순회를 나타낼 수 있다.



풀이 모든 정점들을 한 번씩만 거치면서 순회를 하기 때문에 해밀턴 순회이다.

가중 그래프 (Weighted Graph)

- 가중 그래프
 - 그래프의 각 연결선(edge, arc)에 가중치가 존재
 - 가중치 : 두 정점 사이의 비용 (도로건설비, 차비, 거리, 통신선 구축비, 걸리는 시간 등)
- Weighted Undirected graph
- Weighted Directed graph



〈그림 7.11〉 가중 그래프

평면 그래프 (planar graph)



1010 LAB

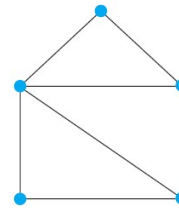
- 평면 그래프 (Planar graph)

- 평면상에 어떠한 연결선도 서로 교차 하지 않는 그래프 (동형 그래프 포함)

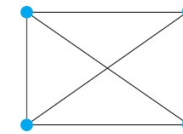


예제 7-10

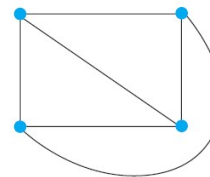
다음 그래프들이 평면 그래프인지의 여부를 알아보자.



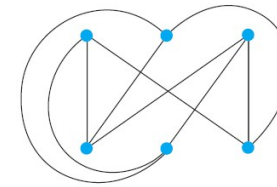
(1)



(2)



(3)



(4)

풀이 (1)은 어떤 연결선들도 서로 교차하지 않으므로 평면 그래프이다. (2)는 (3)과 같이 동형으로 변경할 수 있으므로 평면 그래프이다. 그러나 (4)는 평면 그래프가 아니다.

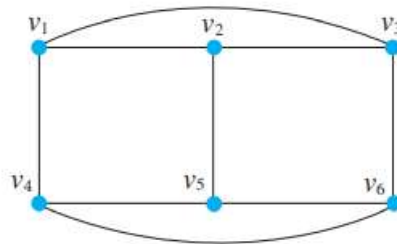
Euler's Formula on planar graphs

- Let G be a connected planar graph with v vertices and e edges. Every planar drawing of G has f faces, where f satisfies $v - e + f = 2$.
 - face (평면): 수를 셀 때, 그래프 바깥의 평면도 포함하여야 함.



예제 7-11

다음 그래프에서 오일러의 정리가 성립하는지를 살펴보자.



풀이 이 그래프에서 정점의 개수 $v = 6$, 연결선의 개수 $e = 9$, 면의 개수 $f = 5$ 이므로 $v - e + f = 2$ 인 오일러의 공식이 성립한다.

평면 그래프와 오일러 공식 예

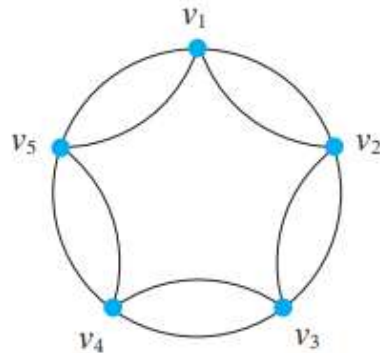


1010 LAB



예제 7-12

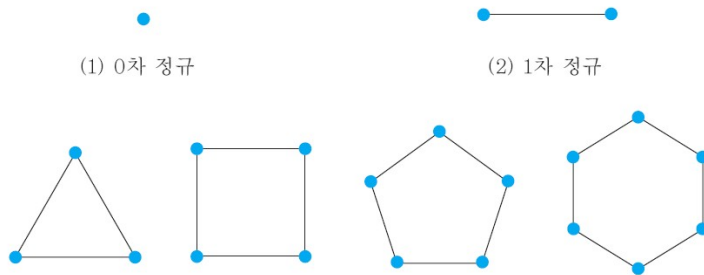
다음 그래프에서 오일러의 공식을 이용하여 오일러의 정리가 성립하는지를 살펴보자.



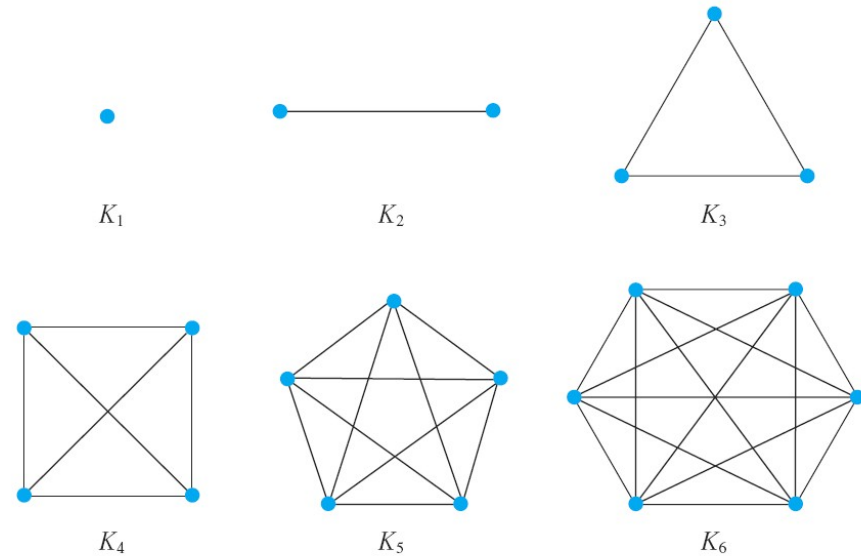
풀이 이 그래프에서 정점의 개수 $v = 5$, 연결선의 개수 $e = 10$, 면의 개수 $f = 7$ 이므로 $v - e + f = 2$ 인 오일러의 공식이 성립한다.

완전 그래프의 연결선 수, 차수, 정규 그래프

- 정규 그래프 (regular graph)
 - a graph where each vertex has the same number of neighbors
- K_n : The complete graph on n vertices
 - the number of edges on $K_n = \frac{n(n-1)}{2}$
 - the degree of all vertices on $K_n = n - 1$
 - $(n - 1)$ 차 정규 그래프



〈그림 7.14〉 k 차 정규 그래프



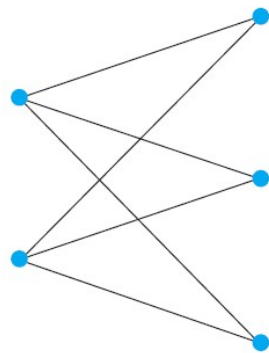
〈그림 7.13〉 완전 그래프의 예

이분 그래프 (bipartite graph)

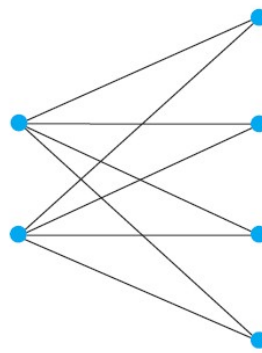
- A bipartite graph $G = (V, E)$ where
 - vertices (V) can be divided into two disjoint and independent sets X and Y ,
 - every edge connects a vertex in X to one in Y .
 - a bipartite graph does not contain any cycles.
- 완전 이분 그래프 (complete bipartite graph)
 - The complete bipartite graph on m and n vertices, denoted by $K_{m,n}$.
 - E connects every vertex in X with all vertices in Y .
 - $K_{m,n}$ has $m \times n$ edges

완전 이분 그래프 예

X 의 정점의 개수가 m 이고, Y 의 정점의 개수가 n 일 때 완전 이분 그래프를 $K_{m,n}$ 으로 나타낸다. 완전 이분 그래프는 <그림 7.15>에 나타나 있다.



(1) $K_{2,3}$

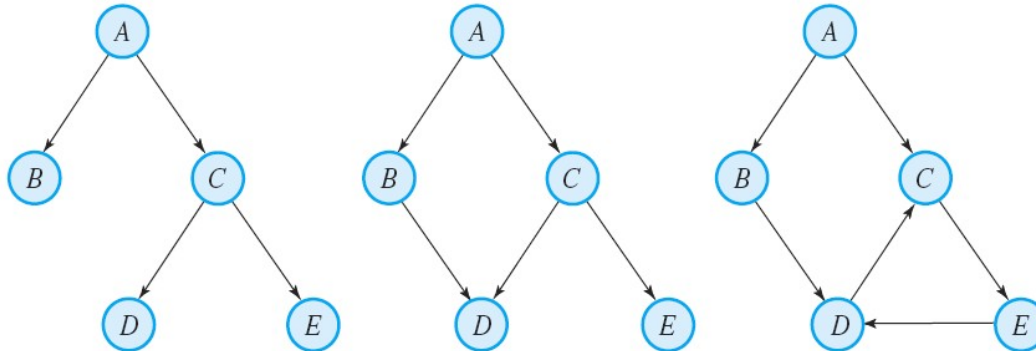


(2) $K_{2,4}$

<그림 7.15> 완전 이분 그래프의 예

방향 비사이클 그래프 (DAG, Directed Acyclic Graph)

- Directed graph with no cycle



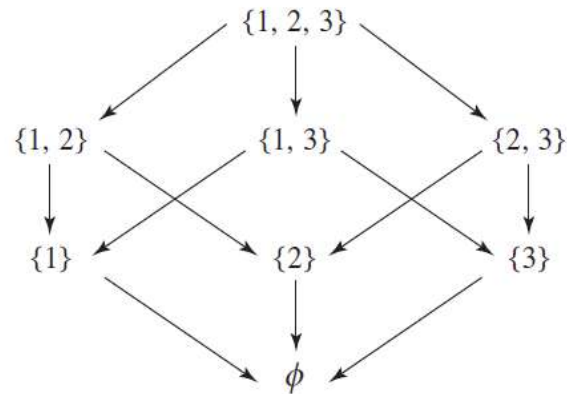
〈그림 7.16〉 트리, DAG, 사이클을 가진 방향 그래프

DAG 예

**예제 7-13**

집합 $\{1, 2, 3\}$ 의 진부분 집합을 DAG로 표현해보자.

풀이 집합 $\{1, 2, 3\}$ 의 진부분 집합 중 2개의 원소로 이루어진 것은 $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$ 이고, 이들의 진부분 집합은 $\{1\}$, $\{2\}$, $\{3\}$ 이다. 또한 ϕ 은 모든 집합의 부분 집합이므로 DAG의 가장 마지막 부분에 위치하게 된다.

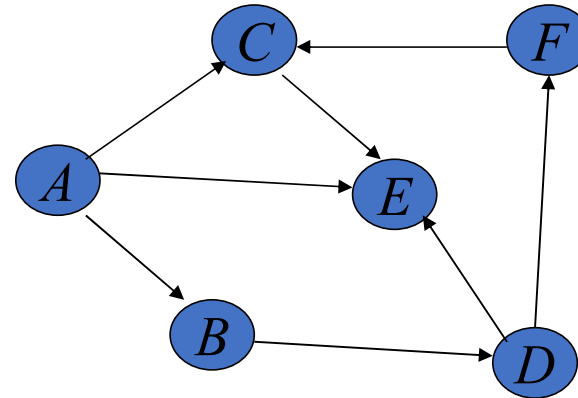
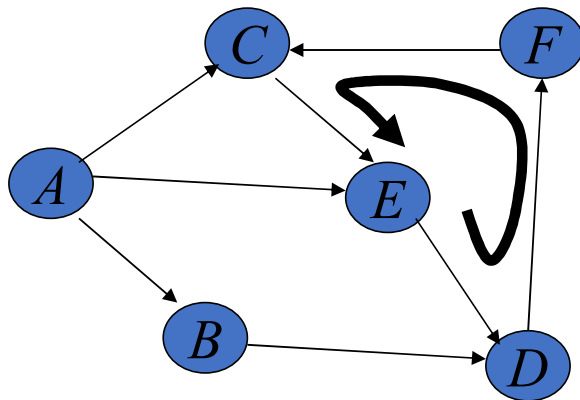


Directed Graphs



1010LAB

- Cycle (in Directed Graph)
- DAG (Directed Acyclic Graph):
 - a directed graph without any cycle

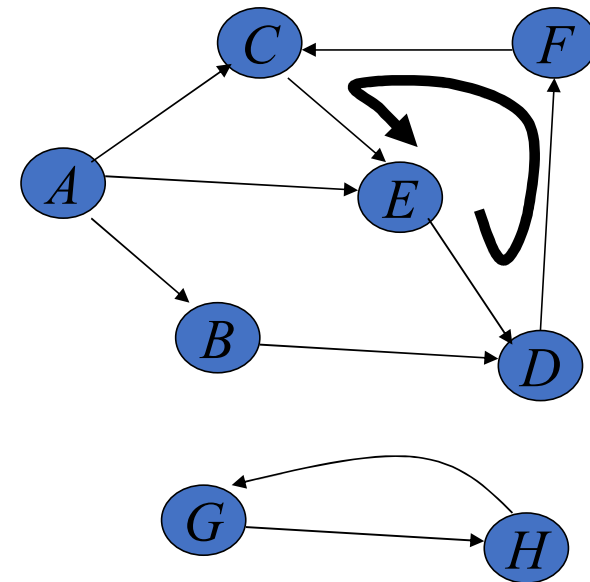


Adjacency Matrix for Directed Graph

- May not be symmetric

	0	1	2	3	4	5	6	7
0	0	1	1	0	1	0	0	0
1	0	0	0	1	0	0	0	0
2	0	0	0	0	1	0	0	0
3	0	0	0	0	0	1	0	0
4	0	0	0	1	0	0	0	0
5	0	0	1	0	0	0	0	0
6	0	0	0	0	0	0	0	1
7	0	0	0	0	0	0	1	0

A : 0
B : 1
C : 2
D : 3
E : 4
F : 5
G : 6
H : 7

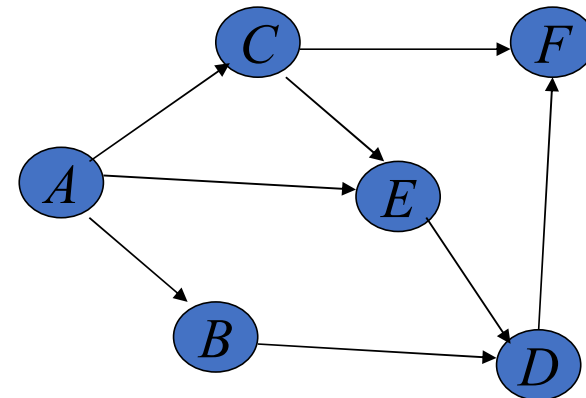


In/out degree



1010 LAB

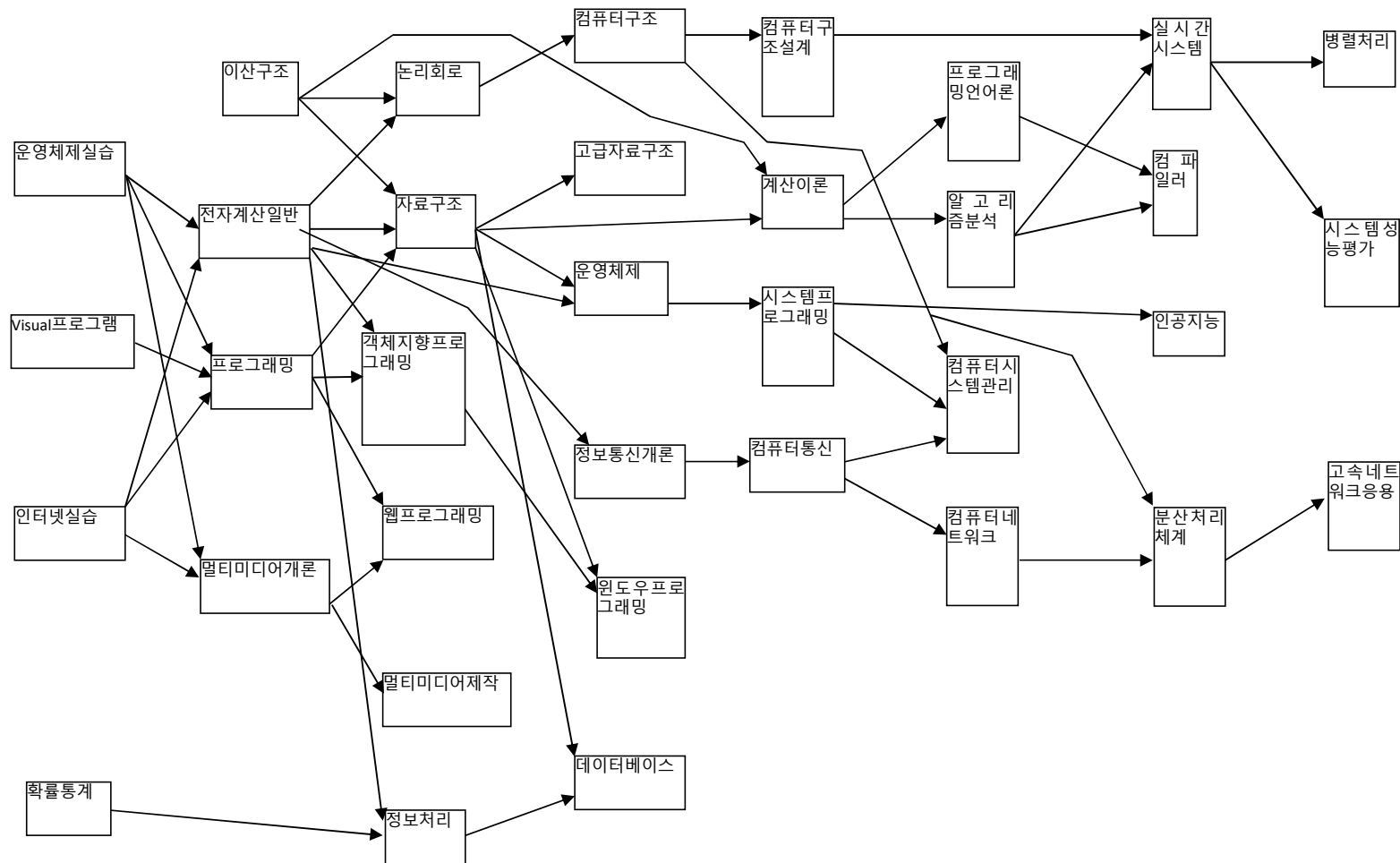
- $G=(V,E)$, a digraph
 - $\text{indegree}(v) = |\{u \in V \mid (u,v) \in E\}|$
 - $\text{outdegree}(v) = |\{u \in V \mid (v,u) \in E\}|$
- **Source (vertex)** : if $\text{indegree}(v) = 0$
- **Sink (vertex)** : if $\text{outdegree}(v) = 0$

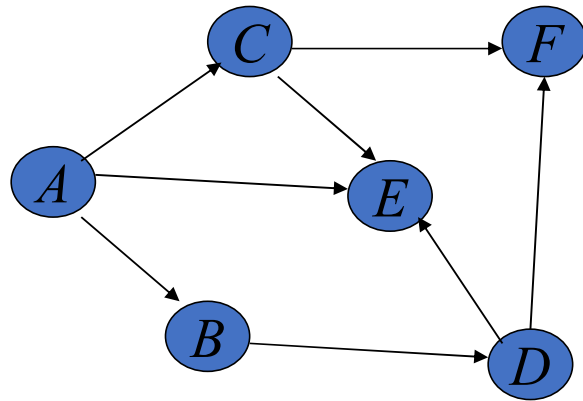


Topological Order

- With Directed Graphs
- Partial ordering
 - irreflexive : $x R x$ 는 성립하지 않음
 - transitive : $x R y$ 이고 $y R z$ 이면, $x R z$ 이다.
 - * cycle이 없다.
- Topological Order
 - a linear ordering of the vertices of a graph such that, for any two vertices, A and B, if A is a predecessor of B in the graph, then A precedes B in the topological order

Topological Order 예 (선수 과목)





Partial order

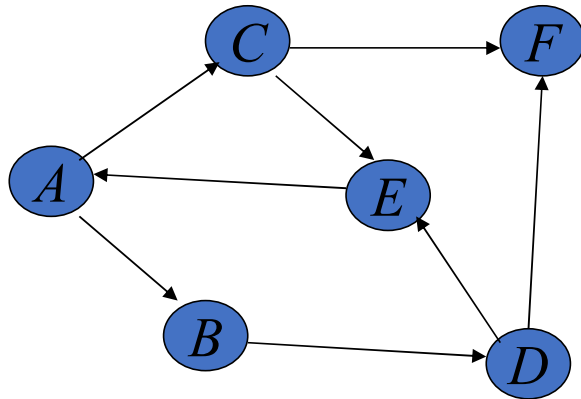
Topological Order

A B C D E F

A C B D F E

Cf.

A B C E D F



Not Partial order

Topological Sorting

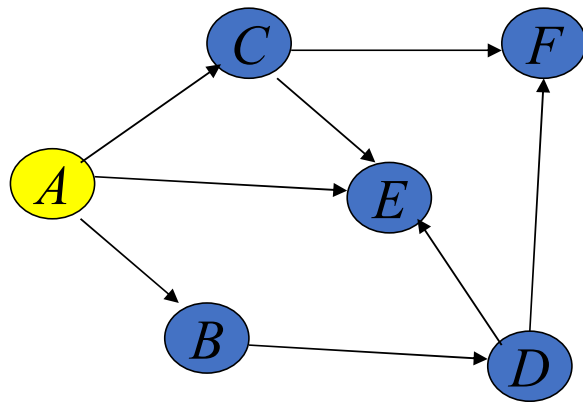


1010 LAB

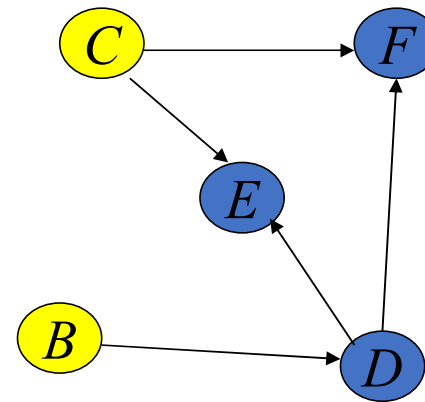
- Finding a topological order

1. If there is a vertex with no predecessor, delete the vertex from the graph (together with all associated edges)
and append it into the list.
2. Repeat 1 until there is no such vertex
3. If the resulting graph has no vertex, the resulting list is a topological order
otherwise, there is no topological order

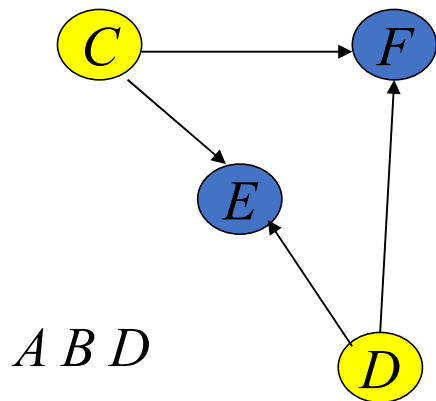
Topological Order 예



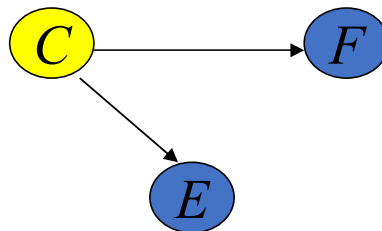
A



A B



A B D



A B D C

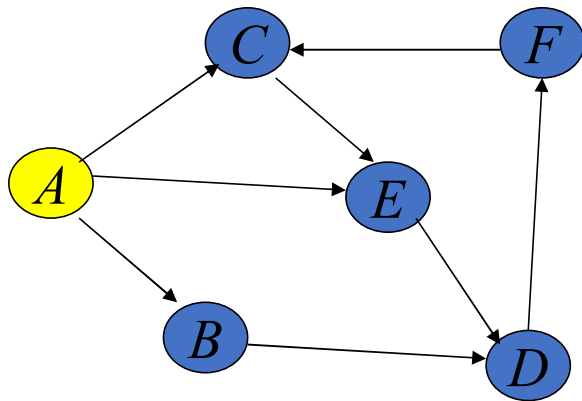


A B D C E

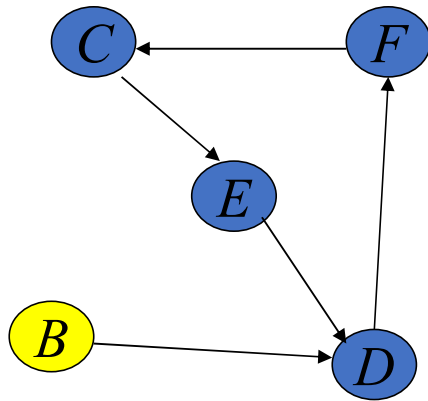


A B D C E F

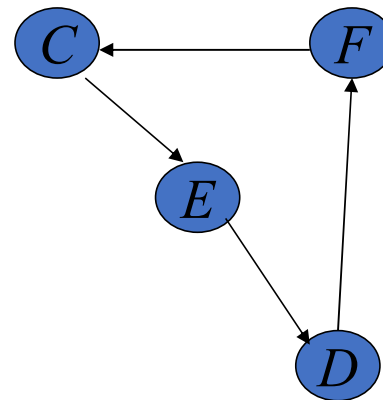
Topological Order 예



A



A B



A B

Stop, there is no topological order

그래프 응용



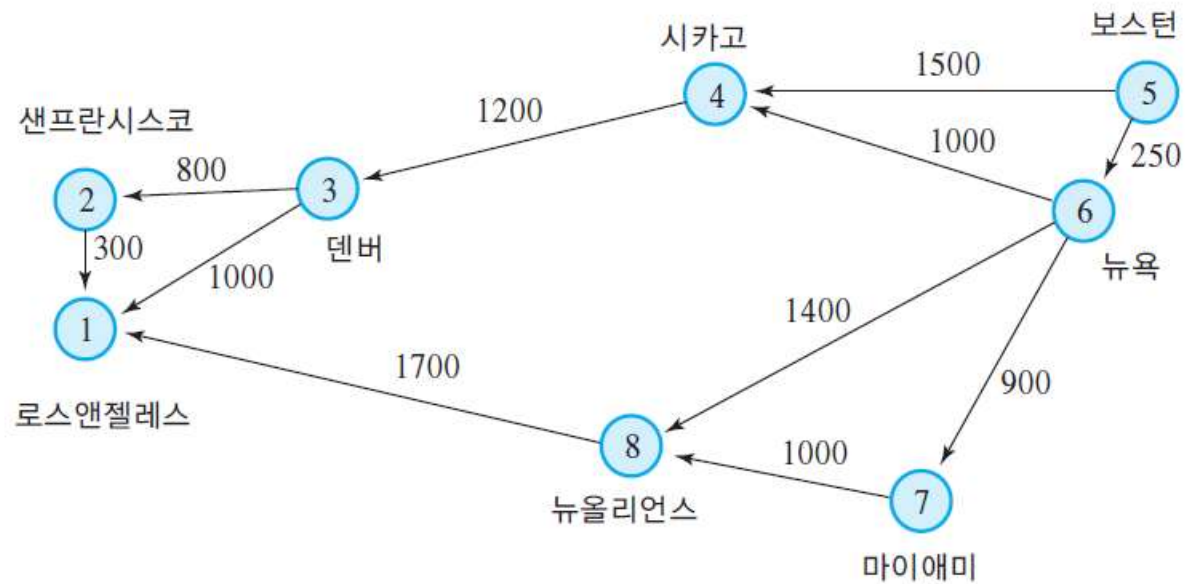
1010 LAB

- 매우 다양함
- 최단 경로 찾기 (Shortest Path Problem)
- 순회 판매원 문제 (Traveling Salesman Problem)

최단 경로 찾기 (Shortest Path Problem)

- 그래프는 지도에서 도시를 나타내는 점과 그들 도시 간의 거리(구축 비용, 차비, 시간 등)를 나타내는 연결선의 값으로 표현됨 → weighted graph
- 도시 A에서 도시 B로 가기 위한 방법 → there may exist many paths having different total costs from A to B
 - 첫째, A에서 B로 가는 경로가 있느냐는 점
 - 둘째, A에서 B로 가는 경로가 여러 개 있을 경우 어떤 경로로 가는 것이 가장 짧은 거리인가 하는 점
- 가장 짧은 거리의 경로를 찾는 문제를 **최단 경로 문제(shortest path problem)**라 함
- 주어진 방향 그래프에서 경로의 시작점을 **출발점(source)**이라 하며 목적지를 **도착점(destination)**이라고 함
- 주어진 연결선의 길이는 0 이상인 경우를 가정함

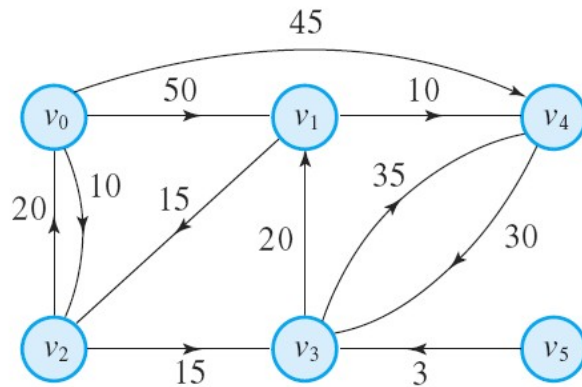
Weighted Digraph 예



〈그림 7.17〉 지도에서의 방향 그래프

최단 경로 예

- 연결선의 값이 두 점 사이의 거리를 나타낼 때, v_0 가 출발점이라 하면 v_0 에서 v_1 까지의 최단 경로는 $v_0v_2v_3v_1$ 임
- 이때 최단 거리는 $10 + 15 + 20 = 45$ 가 됨
- 경로의 길이는 3이지만 v_0 에서 v_1 로 직접 가는 거리인 50보다 짧음
- v_0 에서 v_1, v_2, v_3, v_4 로 가는 최단 경로의 값을 나타냄



〈그림 7.18〉 가중 그래프

	경로	거리
1)	$v_0v_2v_3v_1$	45
2)	v_0v_2	10
3)	$v_0v_2v_3$	25
4)	v_0v_4	45

〈그림 7.19〉 v_0 에서의 최단 경로와 거리

다익스트라 (Dijkstra) 알고리즘

- Single source/All destination 문제 → Dijkstra Algorithm
- All pairs shortest Paths 문제
- **Dijkstra Algorithm**
 - 출발점(source)으로부터 거리가 최소로 알려진 점들의 집합 S 를 유지함으로써 가장 짧은 거리를 가지는 나머지 점 v 를 차례로 S 에 포함시킴.

다익스트라 (Dijkstra) 알고리즘

- 주어진 방향 그래프 $G = (V, E)$ 에서 $v = \{1, 2, \dots, n\}$ 이고 점 $\{1\}$ 이 출발점이라고 가정함
- 점 i 에서 j 로 가는 거리(weight)를 $C[i, j]$ 로 나타내는데, 만약 i 에서 j 로 가는 경로가 없으면 거리는 ∞ 가 됨
- $D[i]$ 는 출발점에서 현재점 i 에 이르는 가장 짧은 거리를 나타냄

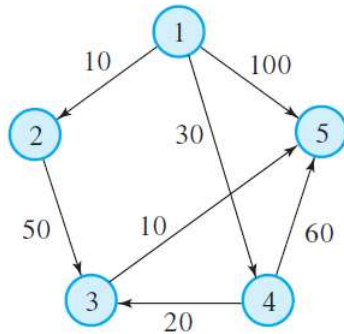
알고리즘 1

다익스트라 알고리즘

```
void Dijkstra()  
    /* 이 알고리즘은 정점 1에서 방향 그래프의 모든 정점으로의 최단 거리를 계산한다. */  
    {  
        S = {1};  
        for( i = 2 ; i <= n ; i++ )  
            D[i] = C[1, i]; /* D값을 초기화한다 */  
        for( i = 1 ; i <= n-1 ; i++ )  
        {  
            choose a vertex w in V - S such that  
                D[w] is a minimum;  
            add w to S;  
            for(each vertex v in V - S)  
                D[v] = min (D[v], D[w] + C[w, v]);  
        }  
    } /* Dijkstra */
```

다익스트라 알고리즘 예

 예제 7-14 다익스트라 알고리즘을 다음과 같은 가중 그래프에 적용해보자.



풀이 처음 단계에서 $S = \{1\}$, $D[2] = 10$, $D[3] = \infty$, $D[4] = 30$, $D[5] = 100$ 이 된다. for 루프의 첫 번째 반복에서 $w = 2$ 가 최소의 D 값을 가진 점으로 선택된다. 그러면 $D[3] = \min(\infty, 10 + 50) = 60$ 이 되고 $D[4]$ 와 $D[5]$ 는 변동이 없다. 왜냐하면 점 1에서 직접 가는 값이 점 2를 경유하는 것보다 값이 작기 때문이다. for 루프에서의 각 반복 과정 후의 D 값은 다음 표와 같으며 4번째 반복 후 점 1에서 점 2, 점 3, 점 4까지의 거리가 각각 10, 50, 30, 60으로 최종 결정된다.

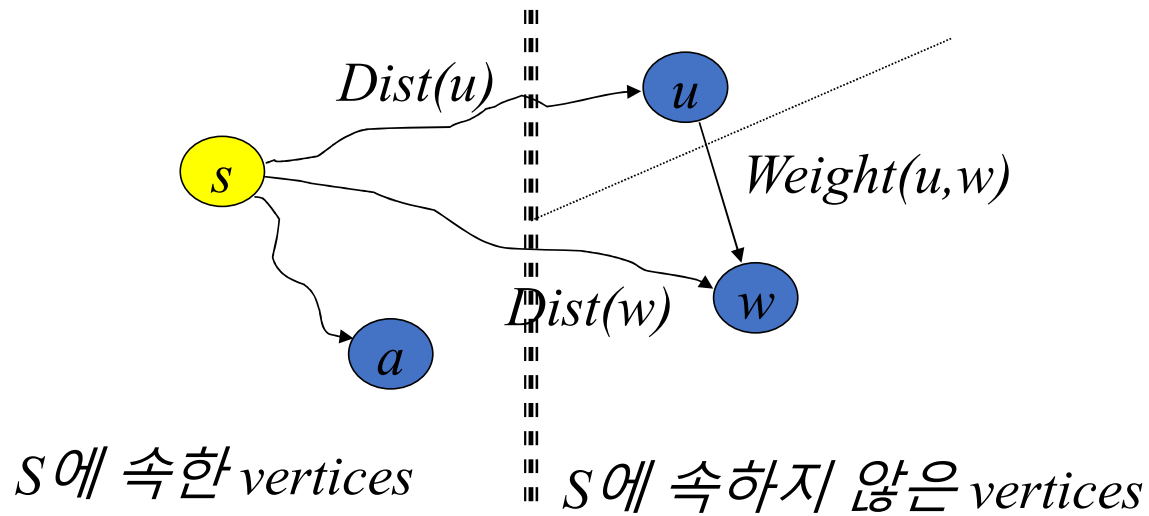
단계	S	w	$D[2]$	$D[3]$	$D[4]$	$D[5]$
0	{1}	—	10	∞	30	100
1	{1, 2}	2	10	60	30	100
2	{1, 2, 4}	4	10	50	30	90
3	{1, 2, 3, 4}	3	10	50	30	60
4	{1, 2, 3, 4, 5}	5	10	50	30	60

다익스트라 알고리즘 다시 한번

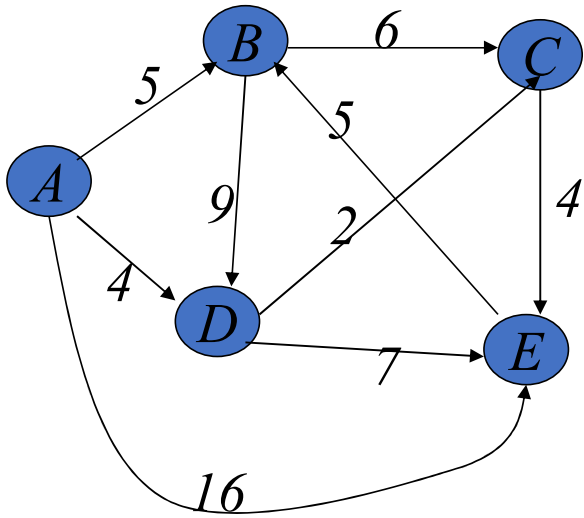
- S : 이미 source vertex(s)로 부터 최단 경로가 알려진 vertex의 집합
 - 초기 : $S = \{s\}$
- $\text{Weight}(u, w)$: vertex u 에서 w 로의 edge의 weight
- $\text{Dist}(w)$:
 - w 가 S 에 속하는 경우:
 - w 가 S 에 속하지 않는 경우: s 에서 S 에 속한 vertex만을 경유하여 w 로 가는 path 중에서 최단 경로의 weight 합
 - s 에서 w 까지의 경로가 아직 알려지지 않은 경우 : 무한대 ∞

• $\text{Dist}(w)$ 의 변경

- w 가 S 에 속하지 않음
- S 에 속하지 않는 u 를 S 에 추가할 때,
 - $\text{Dist}(w)$ 와 $\text{Dist}(u) + \text{weight}(u, w)$ 의 최소값으로 $\text{Dist}(w)$ 를 변경



다익스트라 알고리즘 예



Source A

Step	S	selected vertex (w)	Dist()				
			A	B	C	D	E
초기	{A}	-	0	5		4	16
1	{A}	D	0	5	6	4	11
2	{A,D}	B	0	5	6	4	11
3	{A,B,D}	C	0	5	6	4	10
4	{A,B,C,D}	E	0	5	6	4	10

* 초기/는 $weight(source, u)$

* 각 단계에서 S에 속하지 않은 u 중에서 $Dist(u)$ 가 가장 작은 것을 선택

순회판매원 문제 (Traveling Salesman Problem)

- 순회판매원 문제란 방문해야 할 도시들과 이들 사이의 거리(시간, 비용)가 주어졌을 경우, 순회판매원이 어떤 특정한 도시를 출발하여 어떠한 도시도 두 번 방문함이 없이 모든 도시들을 거쳐 처음 출발한 도시로 되돌아올 때, 총 여행 거리(시간, 비용)가 최소가 되는 경로를 찾는 문제임
- 해밀턴 순회의 응용
- 최소의 경로가 최적의 경로라고 할 수 있음
- 일반적인 해결 알고리즘이 존재하지 않음
- 최근접 이웃 방법(nearest neighbor method)을 통하여 최소값은 아니더라도 근사값은 구할 수 있음 → suboptimal solution
- 임의로 선택한 꼭지점에서 출발하여 그 꼭지점과 가장 가까운 꼭지점을 찾아서 연결하고 있음
- 경로를 첨가하는 과정을 반복하며 마지막에 순회를 형성하도록 하는 것임

순회판매원 문제 정리



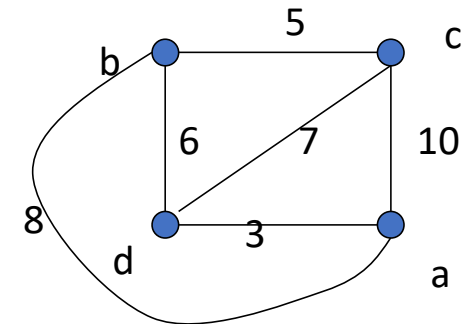
1010 LAB

- **Traveling salesman**

- 방문해야 할 도시(지역)를 가장 싼 비용으로 1번 씩 방문하기 위한
경로 찾기

- **Weighted graph :**

- Edges having their weight
 - weight
 - distance from a city to an adjacent city
 - cost to install a communication line between two cities
 - times taken from a city to an adjacent city



Nearest Neighbor method for Traveling Salesman

- **Suboptimal solution**

begin

Choose any v_1 in V

$v' \leftarrow v_1$

$w \leftarrow 0$

Add v' to the list of vertices in the path

While unmarked vertices remain do

begin

Mark v'

Choose any unmarked vertex, u , that is closest to v'

Add u' to the list of vertices in the path

$w \leftarrow w + \text{the weight of the edge } \{v', u\}$

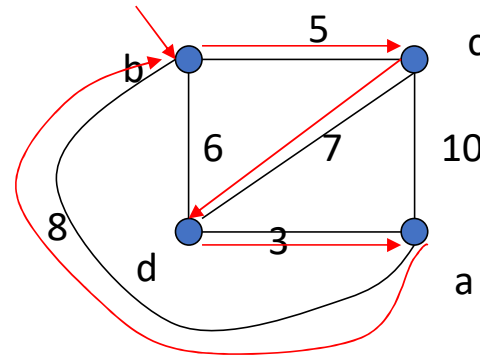
$v' \leftarrow u$

end

Add v_1 to the list of vertices in the path

$w \leftarrow w + \text{the weight of the edge } \{v', v_1\}$

end



b,c,d,a,b

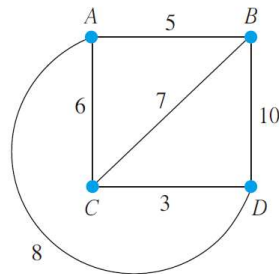
$$w = 5 + 7 + 3 + 8 \\ = 23$$

다시 한번 최근접 이웃 방법

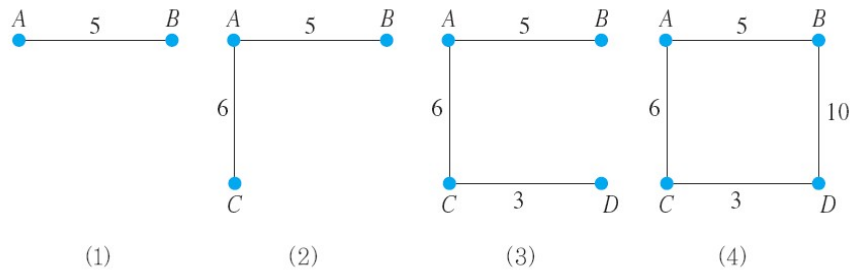


예제 7-15

다음의 가중 그래프에서 최근접 이웃 방법을 적용한 해밀턴 순회를 만들어 보자.

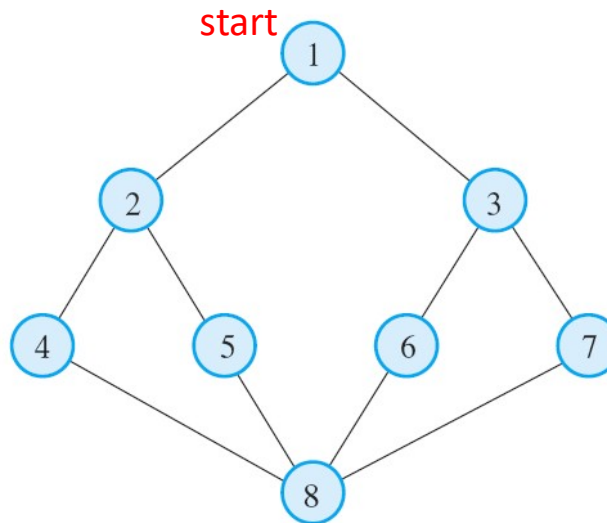


풀이 임의의 정점 B 로부터 시작하자. B 와 가장 가까운 방문되지 않은 정점은 A 이므로 다음 그림 (1)과 같이 A 를 경로에 포함시킨다. 또한 C 를 포함시키고 D 와 B 를 연결한다.



그래프 탐색

- 깊이 우선 탐색 (DFS: Depth First Search)
- 너비 우선 탐색 (BFS: Breadth First Search)



〈그림 7.20〉 주어진 그래프

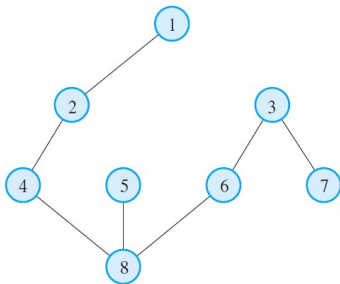
DFS – 깊이 우선 탐색

- 깊이 우선 탐색은 시작점 v 부터 방문함
 - v 에 인접한 정점 중에서 방문하지 않은 정점 w 를 방문하고 다시 w 로부터 탐색을 시작함
 - 어떤 정점 u 를 방문하고 u 에 인접한 모든 정점들을 이미 방문한 경우에는 그 전에 마지막으로 방문한 정점으로 되돌아가서 위의 과정들을 반복함
 - 모든 정점들을 방문한 후 탐색을 종료함
- 순차적인 프로그램보다는 DFS 알고리즘을 재귀(recursive) 알고리즘으로 구현하는 것이 좋음
- 재귀 알고리즘으로 구현할 경우에는 스택(stack)을 사용함
- 스택(stack) : Last-In First-Out (LIFO)

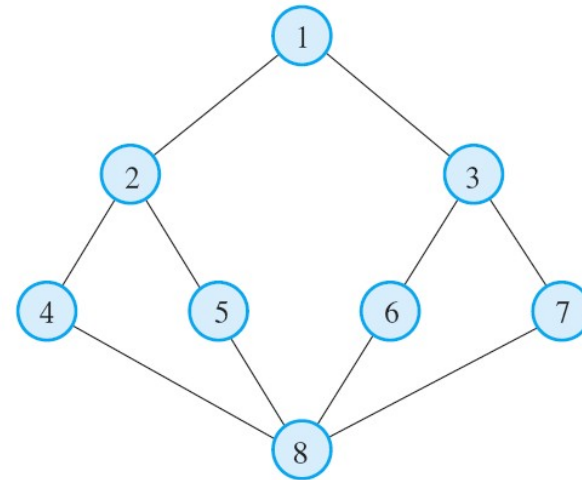
DFS 알고리즘

알고리즘 3 DFS

```
void dfs(int v)
/* G = (V, E)가 n개의 정점을 가진 그래프이고 처음에는 False값으로 행렬
visited[n]이 주어졌다고 할 때, 이 알고리즘은 정점 v로부터 도달 가능한 모든 정
점들을 방문한다. */
{
    int w;
    visited[v] = TRUE;
    for(each vertex w adjacent to v)
        if(!visited[w]) dfs(w);
} /* dfs */
```



〈그림 7.21〉 DFS의 결과



〈그림 7.20〉 주어진 그래프

start: call dfs(1)

visited 1
call dfs(2)

visited 2
call dfs(4)

visited 4
call dfs(8)

visited 8
call dfs(5)

call dfs(6)

visited 5

visited 6
call dfs(3)

visited 3
call dfs(7)

visited 7

call dfs(7)

call dfs(5)

call dfs(3)

BFS – 너비 우선 탐색

- 처음에 방문한 정점과 인접한 정점들을 차례로 방문한다는 점에서 깊이 우선 탐색과 차이가 있음
- 먼저 시작점 v 를 방문한 후 v 에 인접한 모든 정점들을 차례로 방문함
- 더 이상 방문할 정점이 없는 경우 다시 v 에 인접한 정점 가운데 맨 처음으로 방문한 정점과 인접한 정점들을 차례로 방문함
- v 에 인접한 정점 중 두 번째로 방문한 정점과 인접한 정점들을 차례로 방문하는 과정을 반복함
- 모든 정점들을 방문한 후 탐색을 종료함

➤ 깊이 우선 탐색이 스택(stack)을 사용하는데 비해 너비 우선 탐색은 큐(queue)를 사용함

➤ 큐(queue) : First-In First-Out (FIFO)

BFS 알고리즘

```

1 void BFS(int v)
2   let Q be a queue;
3   mark v;
4   Q.enqueue(v);
5   while ( Q is not empty) do {
6       v := Q.dequeue();
9       for ( all edges from v to w in G.adjacentEdges(v) ) do
10           if w is not marked then {
11               mark w;
12               Q.enqueue(w);
13           }
14   }
    
```

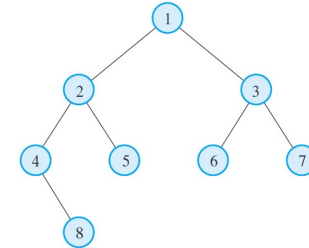
queue



start call BFS(1)
mark 1
enqueue (1)

dequeue(1)

mark 2
enqueue(2)
mark 3
enqueue(3)



〈그림 7.22〉 BFS의 결과

dequeue(2)

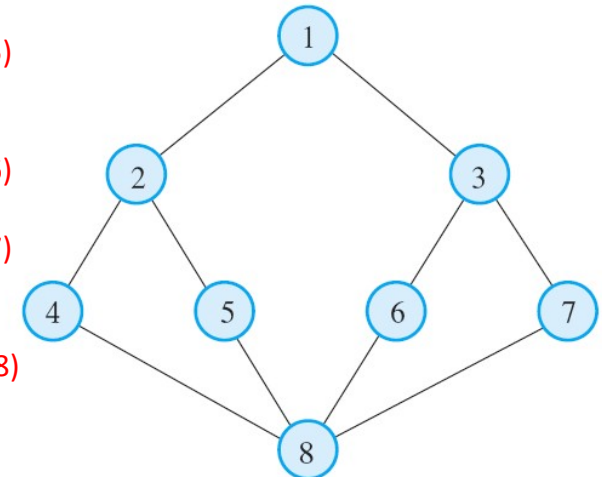
mark 4
enqueue(4)
mark 5
enqueue(5)

dequeue(3)

mark 6
enqueue(6)
mark 7
enqueue(7)

dequeue(4)

mark 8
enqueue(8)



〈그림 7.20〉 주어진 그래프

그래프 색칠 문제 개요

- **vertex coloring**

- a way of coloring the vertices of a graph such that no two adjacent vertices are of the same color
- 단순 그래프의 색칠은 각각의 정점을 서로 다른 색깔로 색칠함으로써 알 수 있음
- 대부분의 그래프에서는 정점의 수보다 적은 색깔을 사용해도 색칠이 가능함
- 그렇다면 몇 가지 색으로 모든 경계가 구별되도록 색칠할 수 있을까?

- **edge coloring**

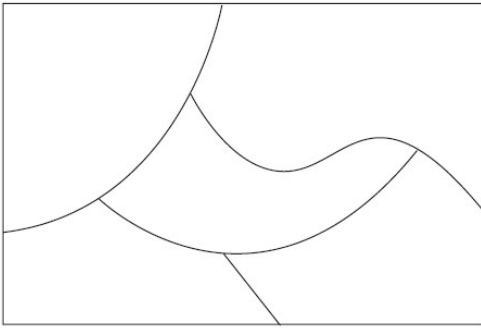
- a color to each edge so that no two adjacent edges are of the same color

지도(map) coloring vs. graph coloring



예제 7-16

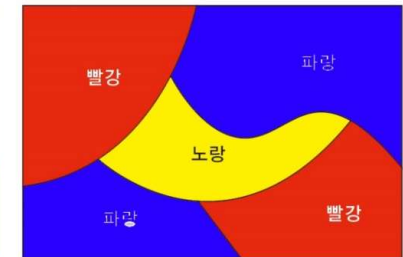
다음 그림과 같이 경계가 주어진 지도를 색칠해보자. 서로 경계가 만나는 부분이 있으면 서로 다른 색을 사용해야 한다.



풀이 다음 그림은 주어진 지도를 5가지의 색을 사용하여 색칠한 예이다.

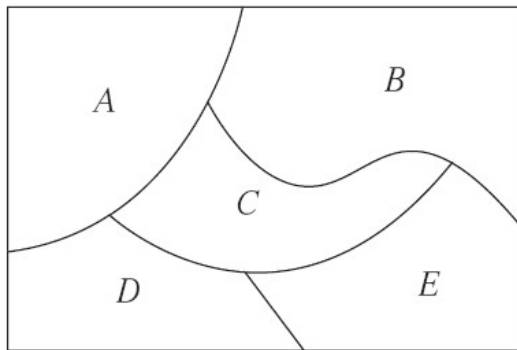


그러나 이 지도는 왼쪽 그림과 같이 4가지 색으로도 가능한 것을 알 수 있으며, 최종적으로 오른쪽 그림과 같이 3가지 색만으로도 가능하다.

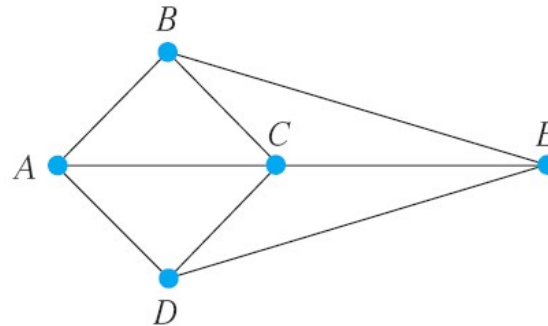


쌍대 그래프

- 지도를 색칠하는 문제는 그래프의 색칠 문제와 연관시킬 수 있음
- 예를 들어, (1)의 그래프와 동형이 되는 (2)의 그래프를 쌍대 그래프라고 함
- 지도의 영역을 색칠하는 문제는 쌍대 그래프의 정점들을 색칠하는 문제와 동일하므로 그래프의 인접한 어떤 정점들도 같은 색깔을 가지지 않도록 해야 함



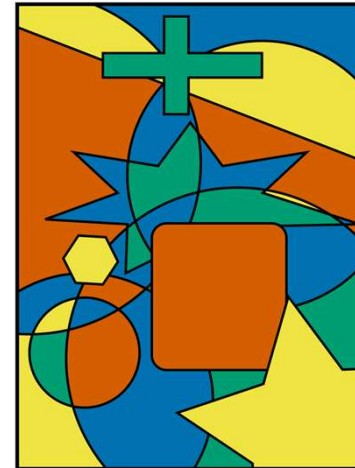
(1)



(2)

Some Theorems on Graph Coloring

- **색칠 수 (chromatic number)** : The smallest number of colors needed to color a graph G
- **Theorem:** The following statements are equivalent.
 - The chromatic number of vertex coloring of a graph G is 2.
 - The graph G is bipartite.
 - The length of every cycle in G is even.
- **Four color theorem:** no more than four colors are required to color the regions of any map so that no two adjacent regions have the same color.
 - the chromatic number of any planar graph is 4.



출처: wikipedia